

Canoga

Canoga is a two-player dice game. This is a variant of the popular Canoga game.

The Objective

The objective of this game is to either cover all one's squares or uncover all the opponent's squares.

The Players

Two players play this game - one player will be the human user of your program, and the other player will be your program/computer. The two players will play a "tournament", consisting of one or more rounds. Each round will consist of the two players playing till one of them wins.

The Setup

The game uses a board with two rows of squares - one for the human player and the other for the opponent. Each row of squares will be numbered 1 through n, where n can be 9, 10 or 11.

A Round

The two players take alternate turns till one of them wins.

A Turn

During one's turn, a player repeatedly throws dice.

- If all the squares 7 through n are covered, the player can choose to throw one die or both dice.
- If any of the squares 7 through n is uncovered, the player must throw both dice.

The player adds the pips of the thrown die/dice. The player must now cover up a set of the player's squares that add up to the sum thrown, that are currently uncovered. For example, if the sum of the two dice is 6, the player can cover the following squares, assuming they are all uncovered at the time:

- 4 and 2
- 5 and 1
- 3, 2 and 1

The player may be able to cover up to 4 squares for one throw of the dice. Alternatively, the player can uncover covered squares of the opponent, subject to the same rules as above. For a given sum of dice, the player must either only cover own squares or only uncover opponent's squares, but not a combination of the two. Eventually, the player throws dice with a total number of pips for which neither of the following is possible:

- no combination of uncovered squares on one's own row;
- no combination of covered squares on opponent's row.

This is when the player's turn ends.

End of Round

A round ends when a player covers all of own squares or uncovers all of opponent's squares. The player is the winner of the round.

Score

If the player wins by covering all of own squares, the winner earns a score equal to the sum of all the uncovered squares of the opponent. If the player wins by uncovering all of opponent's squares, the winner earns a score equal to the sum of all the player's covered squares. (This provides the incentive for both players to cover squares with larger numbers first.)

First Player

Each player throws two dice. The player with the greater sum takes the first turn. Thereafter, the two players alternate. If the sum of both players is the same, dice are thrown repeatedly till a first player can be determined.

Handicap

After the first round, a handicap is placed on the game as follows:

- In the previous round, if the winner was also the one who took the first turn, the opponent gets the advantage;
- In the previous round, if the winner was not the one who took the first turn, the winner gets the advantage.

The advantage is determined as follows: the digits of the winning score are added to a sum in the range 0 through 9. e.g., if the winning score is 27, the sum of 2 + 7 is 9. The next round starts with the corresponding square of the player with the advantage already covered, e.g., the opponent's square 9 is covered if the winner of the previous round took the first turn and vice versa.

The advantage square cannot be uncovered by the opponent until the player with the advantage gets at least one turn in the current round.

The Tournament

After each round, the human player is asked whether she/he wants to play another round.

- If yes, another round is played as described above and the process is repeated.
- If no, the winner of the tournament is announced and the game quits. The winner of the tournament is the player who has won the most number of points, calculated as the sum of the points earned in all the rounds. If both the players earn the same number of points, the tournament is a draw.

Computer Player's Strategy

Your computer player must play to win. It must have a strategy for each of the following:

- Covering its squares;
- Uncovering opponent's squares;
- Whether to cover own squares or uncover opponent's squares after each throw of dice;
- Determining whether to throw one die or two dice when all its squares 7 through n are covered.

Implementation

- **User Interface:** You must provide a user-friendly interface for the game. For C++, LISP and Prolog, ASCII graphics and command-line input are sufficient.
 - The squares must be labeled with their numbers. The player's row and the opponent's row must be clearly marked. Both rows read from left to right.
 - All human inputs must be validated.
 - The turn played by the computer as well as the strategy it uses must be displayed on the screen, e.g.,

The computer tossed the dice 3 and 5.

The sum is 8.

The computer chose to uncover opponent's squares because it had no option to cover its own squares.
It uncovered the following opponent's squares: 4, 3, 1 to maximize the number of squares uncovered.

- **Throwing dice:** Whenever dice are rolled, your program should provide two options:
 1. The program generates the dice randomly;
 2. The program reads the dice as manual input from the user.

This will help you test the game and demonstrate it.

- **Help Mode:** When the human player is playing, the computer must provide a help mode:

- If the human player asks for help, the program must suggest:
 - Whether to cover own squares or uncover opponent's squares and why
 - All the options for covering own squares or uncovering opponent's squares, e.g., "Option 1: Cover 4 and 2. Option 2: Cover 3, 2 and 1"
 - The recommended option and why, e.g., "I recommend that you cover 4 and 2 because.."

The computer must use its own playing strategy to come up with this recommendation.

- **Serialization:** The user should be able to suspend the game after either player has completed a turn, and resume at a later time from where the game was left off. In order to do this:
 - When your program is started, it should provide the option to resume a game from a previously saved state. If yes, it should ask

for the name of the text file from which to read the state of the game, and resume playing from that state.

- During the game, after each player's turn has ended, your program should provide the option to save the game and quit. If the user chooses this option, your program should save the state of the game into a file and quit.

The serialization format for C++/Java Android will be as follows:

Computer:

Squares: 1 2 3 0 5 6 0 8 9 0 11
Score: 34

Human:

Squares: 0 0 3 4 5 0 7 8 0 10 11
Score: 36

First Turn: Computer

Next Turn: Computer

In the above snapshot, computer has covered 4, 7 and 10. Human player has covered 1, 2, 6 and 9. Computer was the first player on this round. It is also the computer's turn next. The sum of the scores from previous rounds is 34 for computer and 36 for human.

Note that the game can be serialized after each turn. So, if a player's turn ends in a win, serialization is done before the win is declared.

In other words, when a game is loaded from a serialization file, the game could immediately declare a winner of the round.

The text format for LISP will be as follows:

```
(
  ; Computer:
  (
    ;Squares:
    ( 1 2 3 0 5 6 0 8 9 0 11 )
    ; Score:
    34
  )

  ; Human:
  (
    ; Squares:
    ( 0 0 3 4 5 0 7 8 0 10 11 )
    ; Score:
    36
  )

  ; First Turn:
  Computer

  ; Next Turn:
  Computer
)
```

Note that the comments above are for your convenience. You do not need to parse those - they will not appear in actual serialization files.

The text format for Prolog will be as follows:

```
[
  % Computer:
  [
    %Squares:
    [ 1 2 3 0 5 6 0 8 9 0 11 ],
    % Score:
    34
  ],

  % Human:
  [
    % Squares:
    [ 0 0 3 4 5 0 7 8 0 10 11 ],
    % Score:
    36
  ],

  % First Turn:
  computer,
```

```
% Next Turn:  
computer  
]
```

Note that the comments above are for your convenience. You do not need to parse those - they will not appear in actual serialization files.

- **Generalization:** At the beginning of each round, the human player must be asked for the size of the rows: A row can be 9, 10 or 11 squares long.
- Using any part of code available in textbooks or on the web is unacceptable.

Assistance

You must meticulously document all the assistance you received, either from another person or from a textbook or online source including generative AI (e.g., ChatGPT). As part of your project demonstration, you will be asked to explain the workings of your code. So, it is important that you understand how any code that you borrow works. For any code that you are unable to explain, you may lose up to 30% of the grade on the project.

Grading

Please see the rubric for some of the features on which your program will be graded.

Milestones

The following are recommended milestones for each project:

- Week 1: A basic, complete game except the items listed for week 2. Attempt the game in the following order:
 1. A turn for human player
 2. A simplistic turn for computer player playing without a strategy, but within the rules of the game
 3. A game loop in which the two players take turns
 4. Deciding the first player
 5. Determining whether the round is over
 6. Determining the winner
 7. Composing all the above within a round
 8. Creating a tournament of multiple rounds and declaring the winner of the tournament
- Week 2: All the strategies for the computer player, help for human player, serialization and generalization.
- Week 3: Code refactoring, cleaning, documentation.